

SC-500 Lab 01

Implement Governance to Enforce Security and Regulatory Compliance

Self-Paced Academic Lab Handout

Verified against Microsoft documentation: 2026-07-13

Lab 01 - Implement Governance to Enforce Security and Regulatory Compliance

Estimated time: 4.5 to 6 hours

Prior labs: None

Study guide subsection: Manage identity, access, and governance > Implement governance to enforce security and regulatory compliance

Coverage map

This lab covers every item in the official SC-500 subsection:

1. Implement and configure security controls by using Azure Policy, including built-in and custom policy definitions.
2. Evaluate regulatory compliance by using Microsoft Defender for Cloud.
3. Implement and configure security controls in Defender for Cloud, including security standards and recommendations.
4. Implement resource locks.
5. Manage Azure built-in role assignments.
6. Manage custom roles, including Azure roles and Microsoft Entra roles.
7. Evaluate and remediate overprivileged access assignments by using Azure role-based access control.
8. Configure security controls for backup protection by using Azure Backup security features.
9. Implement and configure security controls by using infrastructure as code.

Prerequisites

Azure and Microsoft Entra access

- A dedicated non-production Azure subscription.
- **Owner** on the lab subscription is the simplest option. A narrower combination is acceptable only if it includes resource deployment, policy definition and assignment, role assignment, and lock-management permissions.
- A Microsoft Entra test user. Do not use a production administrator as the test principal.
- For the Microsoft Entra custom-role exercise:
 - Microsoft Entra ID P1 or P2.
 - Privileged Role Administrator.
 - Microsoft Graph PowerShell.
- A supported PowerShell 7 release.
- A current Azure CLI release.
- VS Code with the Bicep extension, or another text editor.
- Bicep CLI through Azure CLI.
- Permission to open the Azure portal, Microsoft Entra admin center, and Microsoft Defender portal.

Cost boundary

This lab intentionally creates no virtual machines, databases, gateways, or paid Defender workload plans.

Expected charge is negligible:

- One empty Standard LRS storage account.
- One user-assigned managed identity.
- One empty Recovery Services vault.
- One Resource Guard.
- Azure Policy, Azure RBAC, resource locks, and custom role definitions do not create usage charges.
- Foundational CSPM is free and is enabled with Defender for Cloud.

Do not enable Defender CSPM or any paid Defender workload plan during this lab. Some advanced standards, governance rules, attack paths, and regulatory features depend on Defender CSPM. The lab uses the Microsoft Cloud Security Benchmark and free recommendations so the exercise remains suitable for a personal pay-as-you-go subscription.

Safety rules

- Use a dedicated resource group and a unique prefix.
- Do not assign the deny policy outside the lab resource group.
- Do not lock the Recovery Services vault's immutability state. A locked immutable vault cannot be reverted.
- Do not protect any VM, database, file share, or other workload with the lab vault.
- Complete Section 5 before ending the lab.

Learning objectives

By the end of the lab, you will be able to:

1. Deploy an Azure governance baseline through Bicep.
2. Validate and preview an infrastructure-as-code deployment before changing Azure.
3. Author, assign, evaluate, and remediate Azure Policy definitions.
4. Distinguish Audit, Deny, and Modify effects.
5. Apply and test CanNotDelete and ReadOnly resource locks.
6. Assign Azure built-in roles at appropriate scopes.
7. Build and assign a constrained Azure custom role.
8. Build and assign a scoped Microsoft Entra custom role.
9. identify and remove an overprivileged Azure role assignment.
10. Evaluate Microsoft Cloud Security Benchmark controls and Defender for Cloud recommendations.
11. Configure Recovery Services vault soft delete, reversible immutability, and multi-user authorization with Resource Guard.
12. Remove every resource and assignment created by the lab.

1. Implementation walkthrough

1.1 Prepare the local workspace and Azure context

Perform the implementation from a PowerShell 7 terminal. The commands invoke Azure CLI unless otherwise stated.

1. Create a local working directory.

```
New-Item -ItemType Directory -Path "$HOME\sc500-lab01" -Force | Out-Null
Set-Location "$HOME\sc500-lab01"
```

2. Sign in with device code authentication.

```
az login --use-device-code
```

3. List subscriptions and set the lab subscription explicitly.

```
az account list --query "[].{Name:name,Id:id,State:state,Default:isDefault}" -o table

$SubscriptionId = "<PASTE-LAB-SUBSCRIPTION-ID>"
az account set --subscription $SubscriptionId
az account show --query "{Name:name,SubscriptionId:id,TenantId:tenantId,User:user.name}" -o table
```

4. Define lab variables. The prefix must contain only lowercase letters and numbers.

```
$Prefix = "sc500$((Get-Random -Maximum 9999).ToString('0000'))"
$Location = "canadacentral"
$DeploymentName = "$Prefix-baseline"

$RG = "$Prefix-gov-rg"
$GuardRG = "$Prefix-guard-rg"
$IdentityName = "$Prefix-deny-test"
$VaultName = "$Prefix-rsv"
$ResourceGuardName = "$Prefix-guard"

$TagAssignmentName = "$Prefix-addtag"
$DenyAssignmentName = "$Prefix-reqtag"
$TagRemediationName = "$Prefix-tag-remediation"

$AzureCustomRoleName = "SC500 $Prefix Governance Auditor"
$EntraCustomRoleName = "SC500 $Prefix App Basic Administrator"

$TestUserUpn = "<TEST-USER-UPN>"
```

5. Capture subscription, tenant, and administrator object IDs.

```
$SubscriptionId = az account show --query id -o tsv
$TenantId = az account show --query tenantId -o tsv
$CurrentUserObjectId = az ad signed-in-user show --query id -o tsv

$SubscriptionScope = "/subscriptions/$SubscriptionId"

[pscustomobject]@{
    Prefix = $Prefix
    SubscriptionId = $SubscriptionId
    TenantId = $TenantId
    CurrentUserObjectId = $CurrentUserObjectId
    TestUserUpn = $TestUserUpn
} | Format-List
```

6. Confirm that the test user exists.

```
$TestId = az ad user show --id $TestUserUpn --query id -o tsv

if (-not $TestId) {
    throw "Test user not found. Correct `$TestUserUpn before continuing."
}

$TestId
```

7. Register the resource providers used by the lab.

```
$Providers = @(
    "Microsoft.Authorization",
    "Microsoft.Insights",
    "Microsoft.ManagedIdentity",
    "Microsoft.PolicyInsights",
    "Microsoft.RecoveryServices",
    "Microsoft.Resources",
    "Microsoft.Security",
    "Microsoft.Storage",
    "Microsoft.DataProtection"
```

```

)
foreach ($Provider in $Providers) {
    az provider register --namespace $Provider --wait
}

foreach ($Provider in $Providers) {
    az provider show `
        --namespace $Provider `
        --query "{Provider:namespace,State:registrationState}" `
        -o table
}

```

8. Install or update Bicep.

```

az bicep install
az bicep upgrade
az bicep version

```

Checkpoint 1

Expected results:

- az account show identifies the intended subscription and tenant.
- \$TestUserId contains a GUID.
- The required resource providers show Registered.
- az bicep version returns a version without an error.

Do not continue if the active subscription is a production subscription.

1.2 Author the baseline as Bicep

The deployment intentionally creates a storage account with shared-key access enabled. A custom Audit policy will identify it as non-compliant. You will remediate the setting later.

1. Create workload.bicep.

```

@'
targetScope = 'resourceGroup'

@description('Lowercase alphanumeric lab prefix.')
param prefix string

@description('Deployment region.')
param location string

var storageName = take('${toLower(prefix)}${uniqueString(resourceGroup().id)}', 24)

resource storage 'Microsoft.Storage/storageAccounts@2023-05-01' = {
    name: storageName
    location: location
    tags: {
        Environment: 'Lab'
        Workload: 'SC-500'
        DataClassification: 'Internal'
    }
    sku: {
        name: 'Standard_LRS'
    }
    kind: 'StorageV2'
    properties: {
        accessTier: 'Hot'
        allowBlobPublicAccess: false
        allowCrossTenantReplication: false
        allowSharedKeyAccess: true
        defaultToOAuthAuthentication: true
        enableHttpsTrafficOnly: true
        minimumTlsVersion: 'TLS_1_2'
    }
}

```

```

    publicNetworkAccess: 'Enabled'
  encryption: {
    keySource: 'Microsoft.Storage'
    requireInfrastructureEncryption: true
    services: {
      blob: {
        enabled: true
      }
      file: {
        enabled: true
      }
    }
  }
  networkAcls: {
    bypass: 'AzureServices'
    defaultAction: 'Allow'
  }
}
}
}

output storageAccountName string = storage.name
output storageAccountId string = storage.id
"@ | Set-Content -Path ".\workload.bicep" -Encoding utf8

```

2. Create main.bicep.

```

@'
targetScope = 'subscription'

@minLength(5)
@maxLength(12)
@description('Lowercase alphanumeric lab prefix.')
param prefix string

@description('Deployment region.')
param location string = 'canadacentral'

var resourceGroupName = '${prefix}-gov-rg'
var policyDefinitionName = take('${prefix}-audit-shared-key', 64)
var policyAssignmentName = take('${prefix}-pa-shared-key', 24)

resource labRg 'Microsoft.Resources/resourceGroups@2024-03-01' = {
  name: resourceGroupName
  location: location
  tags: {
    Environment: 'Lab'
    Workload: 'SC-500'
    Owner: 'Student'
    DataClassification: 'Internal'
  }
}

module workload './workload.bicep' = {
  name: '${prefix}-workload'
  scope: labRg
  params: {
    prefix: prefix
    location: location
  }
}

resource auditSharedKey 'Microsoft.Authorization/policyDefinitions@2023-04-01' = {
  name: policyDefinitionName
  properties: {
    policyType: 'Custom'
    mode: 'Indexed'
    displayName: 'SC-500 - Audit storage accounts that permit shared-key access'
    description: 'Audits storage accounts unless shared-key authorization is explicitly disabled.'
    metadata: {
      category: 'Storage'
      version: '1.0.0'
      source: 'SC-500 Lab 01'
    }
  }
}

```

```

    }
    policyRule: {
      if: {
        allOf: [
          {
            field: 'type'
            equals: 'Microsoft.Storage/storageAccounts'
          }
          {
            field: 'Microsoft.Storage/storageAccounts/allowSharedKeyAccess'
            notEquals: false
          }
        ]
      }
      then: {
        effect: 'audit'
      }
    }
  }
}

resource auditSharedKeyAssignment 'Microsoft.Authorization/policyAssignments@2024-05-01' = {
  name: policyAssignmentName
  scope: labRg
  properties: {
    displayName: 'SC-500 - Audit storage shared-key access'
    description: 'Lab assignment for the custom storage authorization policy.'
    policyDefinitionId: auditSharedKey.id
    enforcementMode: 'Default'
    nonComplianceMessages: [
      {
        message: 'Shared-key access must be disabled. Use Microsoft Entra authorization and narrowly
        scoped SAS where required.'
      }
    ]
  }
}

output resourceGroupName string = labRg.name
output resourceGroupId string = labRg.id
output storageAccountName string = workload.outputs.storageAccountName
output storageAccountId string = workload.outputs.storageAccountId
output customPolicyDefinitionId string = auditSharedKey.id
output customPolicyAssignmentId string = auditSharedKeyAssignment.id
"@ | Set-Content -Path ".\main.bicep" -Encoding utf8

```

3. Build the Bicep files.

```
az bicep build --file ".\main.bicep"
```

Expected result: main.json is created and the command returns no compilation error.

4. Validate the subscription deployment.

```

az deployment sub validate `
  --name $DeploymentName `
  --location $Location `
  --template-file ".\main.bicep" `
  --parameters prefix=$Prefix location=$Location `
  -o table

```

Expected result: provisioningState is Succeeded.

5. Preview the change set before deployment.

```

az deployment sub what-if `
  --name $DeploymentName `
  --location $Location `
  --template-file ".\main.bicep" `
  --parameters prefix=$Prefix location=$Location

```

Expected result: the preview shows creation of:

- One resource group.
- One storage account.
- One custom policy definition.
- One custom policy assignment.

Review the scope of each item. The assignment must be scoped to the lab resource group, not the subscription.

6. Deploy.

```
az deployment sub create `
--name $DeploymentName `
--location $Location `
--template-file ".\main.bicep" `
--parameters prefix=$Prefix location=$Location `
-o table
```

7. Capture deployment outputs.

```
$Outputs = az deployment sub show `
--name $DeploymentName `
--query properties.outputs `
-o json | ConvertFrom-Json

$RG = $Outputs.resourceGroupName.value
$RgId = $Outputs.resourceGroupId.value
$StorageName = $Outputs.storageAccountName.value
$StorageId = $Outputs.storageAccountId.value
$CustomPolicyDefinitionId = $Outputs.customPolicyDefinitionId.value
$CustomPolicyAssignmentId = $Outputs.customPolicyAssignmentId.value

$CustomPolicyDefinitionName = Split-Path $CustomPolicyDefinitionId -Leaf
$CustomPolicyAssignmentName = Split-Path $CustomPolicyAssignmentId -Leaf

[pscustomobject]@{
    ResourceGroup = $RG
    StorageAccount = $StorageName
    PolicyDefinition = $CustomPolicyDefinitionName
    PolicyAssignment = $CustomPolicyAssignmentName
} | Format-List
```

8. Verify security-relevant storage settings.

```
az storage account show `
--resource-group $RG `
--name $StorageName `
--query "{
    HttpsOnly:enableHttpsTrafficOnly,
    MinimumTls:minimumTlsVersion,
    BlobPublicAccess:allowBlobPublicAccess,
    SharedKeyAccess:allowSharedKeyAccess,
    DefaultToOAuth:defaultToOAuthAuthentication,
    InfrastructureEncryption:encryption.requireInfrastructureEncryption,
    PublicNetworkAccess:publicNetworkAccess
}" `
-o table
```

Expected result:

- HTTPS only: True
- Minimum TLS: TLS1_2
- Blob public access: False
- Shared-key access: True - deliberately non-compliant
- Default to OAuth: True
- Infrastructure encryption: True

9. Portal verification:

- Open **Azure portal** > **Resource groups** > <RG>.
- Confirm the storage account is present.
- Open **Subscriptions** > <subscription> > **Deployments**.
- Open the deployment named in \$DeploymentName.
- Review **Inputs**, **Outputs**, and **Template**.
- Open **Policy** > **Definitions**, filter **Type** = **Custom**, and locate the policy.
- Open **Policy** > **Assignments**, filter to the lab resource group, and locate the assignment.

Checkpoint 2

Save the following evidence:

- The what-if result.
- The deployment output table.
- The storage security-settings table.
- The custom policy definition and assignment IDs.

1.3 Evaluate and remediate Azure Policy

Azure Policy evaluates resources against assigned definitions. Audit reports a finding. Deny blocks an operation. Modify can change supported properties or tags and uses a managed identity for remediation of existing resources.

1.3.1 Evaluate the custom Audit policy

1. Trigger an on-demand compliance scan.

```
az policy state trigger-scan --resource-group $RG
```

2. Wait two to five minutes, then query the assignment state.

```
az policy state list `
--resource-group $RG `
--filter "PolicyAssignmentId eq '$CustomPolicyAssignmentId'" `
--query "[].{
  Resource:resourceId,
  Compliance:complianceState,
  Policy:policyDefinitionName,
  Timestamp:timestamp
}" `
-o table
```

If no row appears, wait five minutes and rerun the scan and query. Initial policy evaluation is asynchronous.

Expected result: the storage account is NonCompliant.

3. Portal verification:

- Open **Azure portal** > **Policy** > **Compliance**.
- Set **Scope** to the lab resource group.
- Select the custom assignment.
- Open **Resource compliance**.
- Confirm that the storage account is non-compliant and review the non-compliance message.

1.3.2 Assign a built-in Modify policy and remediate an existing resource

1. Locate the built-in definition by display name.

```
$TagModifyPolicyId = az policy definition list `
--query "[?displayName=='Add or replace a tag on resources'].id | [0]" `
-o tsv
```



```
if (-not $TagModifyPolicyId) {
    throw "Built-in policy not found. Search Azure Policy Definitions for 'Add or replace a tag on
    resources' and set `$TagModifyPolicyId to its resource ID."
}

$TagModifyPolicyId
```

2. Write the parameter file.

```
@'
{
  "tagName": {
    "value": "GovernedBy"
  },
  "tagValue": {
    "value": "SC-500-Lab"
  }
}
'@ | Set-Content -Path ".\tag-parameters.json" -Encoding utf8
```

3. Assign the policy at resource-group scope with a system-assigned managed identity. The CLI also assigns the minimum lab role specified here to the policy identity.

```
az policy assignment create `
  --name $TagAssignmentName `
  --display-name "SC-500 - Add governance tag" `
  --description "Adds or replaces GovernedBy=SC-500-Lab on resources in the lab resource group." `
  --scope $RgId `
  --policy $TagModifyPolicyId `
  --params "@tag-parameters.json" `
  --mi-system-assigned `
  --location $Location `
  --identity-scope $RgId `
  --role "Tag Contributor" `
  --non-compliance-messages "[{"message`":`"The GovernedBy tag is missing or incorrect.`"}]" `
  -o jsonc
```

4. Capture the assignment ID and identity principal ID.

```
$TagAssignmentId = az policy assignment show `
  --name $TagAssignmentName `
  --scope $RgId `
  --query id -o tsv

$TagPolicyIdentityId = az policy assignment show `
  --name $TagAssignmentName `
  --scope $RgId `
  --query identity.principalId -o tsv

[pscustomobject]@{
    AssignmentId = $TagAssignmentId
    ManagedIdentityPrincipalId = $TagPolicyIdentityId
} | Format-List
```

5. Verify the managed identity role assignment.

```
az role assignment list `
  --assignee-object-id $TagPolicyIdentityId `
  --scope $RgId `
  --all `
  --query "[].{Role:roleDefinitionName,Scope:scope,PrincipalType:principalType}" `
  -o table
```

Expected result: Tag Contributor at the lab resource-group scope.

6. Create a remediation task for resources that already exist.

```
az policy remediation create `
  --resource-group $RG `
  --name $TagRemediationName `
```

```
--policy-assignment $TagAssignmentId `
--resource-discovery-mode ReEvaluateCompliance `
-o jsonc
```

7. Monitor the task.

```
do {
    Start-Sleep -Seconds 15
    $RemediationState = az policy remediation show `
        --resource-group $RG `
        --name $TagRemediationName `
        --query provisioningState -o tsv

    Write-Host "Remediation state: $RemediationState"
} while ($RemediationState -in @("Accepted", "Evaluating", "Running"))
```

Expected result: Succeeded or Complete.

8. Verify the tag on the storage account.

```
az resource show `
--ids $StorageId `
--query tags `
-o jsonc
```

Expected result includes:

```
"GovernedBy": "SC-500-Lab"
```

1.3.3 Assign and test a built-in Deny policy

1. Locate the built-in definition.

```
$RequireTagPolicyId = az policy definition list `
--query "[?displayName=='Require a tag and its value on resources'].id | [0]" `
-o tsv

if (-not $RequireTagPolicyId) {
    throw "Built-in policy not found. Search Azure Policy Definitions for 'Require a tag and its value on resources' and set `$RequireTagPolicyId."
}
```

2. Write the parameter file.

```
@'
{
  "tagName": {
    "value": "DataClassification"
  },
  "tagValue": {
    "value": "Internal"
  }
}
'@ | Set-Content -Path ".\deny-tag-parameters.json" -Encoding utf8
```

3. Assign the definition only to the lab resource group.

```
az policy assignment create `
--name $DenyAssignmentName `
--display-name "SC-500 - Require Internal data classification" `
--description "Denies resource creation when DataClassification is not Internal." `
--scope $RgId `
--policy $RequireTagPolicyId `
--params "@deny-tag-parameters.json" `
--non-compliance-messages "[{"message\":\"Set DataClassification=Internal before deployment.\"}]" `
-o jsonc
```

4. Wait two minutes for assignment propagation.

```
Start-Sleep -Seconds 120
```

5. Attempt to create a user-assigned identity without the required tag.

```
az identity create `
  --resource-group $RG `
  --name $IdentityName `
  --location $Location `
  -o table
```

Expected result: the request fails with `RequestDisallowedByPolicy` or an equivalent policy-denial message.

If the identity is created because policy propagation has not completed:

```
az identity delete --resource-group $RG --name $IdentityName
Start-Sleep -Seconds 180
```

Then repeat the denied request.

6. Create the identity with the required tag.

```
az identity create `
  --resource-group $RG `
  --name $IdentityName `
  --location $Location `
  --tags `
    Environment=Lab `
    Workload=SC-500 `
    DataClassification=Internal `
  -o table
```

Expected result: `provisioningState` is `Succeeded`.

7. Confirm both policies affected the resource.

```
az identity show `
  --resource-group $RG `
  --name $IdentityName `
  --query "{Name:name,PrincipalId:principalId,Tags:tags}" `
  -o jsonc
```

Expected result:

- `DataClassification` is `Internal`.
- `GovernedBy` is added or remediated after policy evaluation.

8. Trigger another scan and view current compliance.

```
az policy state trigger-scan --resource-group $RG
Start-Sleep -Seconds 120

az policy state summarize `
  --resource-group $RG `
  -o jsonc
```

1.3.4 Remediate the custom storage policy

1. Disable shared-key authorization.

```
az storage account update `
  --resource-group $RG `
  --name $StorageName `
  --allow-shared-key-access false `
  -o table
```

2. Trigger policy evaluation.

```
az policy state trigger-scan --resource-group $RG
Start-Sleep -Seconds 120
```

3. Re-query the custom assignment.

```
az policy state list `
--resource-group $RG `
--filter "PolicyAssignmentId eq '$CustomPolicyAssignmentId'" `
--query "[].{Resource:resourceId,Compliance:complianceState,Timestamp:timestamp}" `
-o table
```

Expected result: the storage account changes to Compliant.

Checkpoint 3

Record:

- Custom Audit policy: NonCompliant before the change, Compliant afterward.
- Modify policy: existing storage account received GovernedBy=SC-500-Lab.
- Deny policy: untagged identity creation was blocked.
- Tagged identity creation succeeded.
- Modify assignment has a system-assigned identity with Tag Contributor only at the lab resource-group scope.

1.4 Implement and test resource locks

A CanNotDelete lock permits authorized updates but blocks deletion. A ReadOnly lock permits reads but blocks updates and deletion. Locks inherit to child resources.

1. Capture the managed identity resource ID.

```
$IdentityId = az identity show `
--resource-group $RG `
--name $IdentityName `
--query id -o tsv
```

2. Create a CanNotDelete lock on the managed identity.

```
az lock create `
--name "sc500-cannot-delete" `
--lock-type CanNotDelete `
--resource-group $RG `
--resource $IdentityName `
--resource-type "Microsoft.ManagedIdentity/userAssignedIdentities" `
--notes "SC-500 Lab 01 deletion test" `
-o table
```

3. Attempt to delete the identity.

```
az identity delete --resource-group $RG --name $IdentityName
```

Expected result: the request fails with ScopeLocked or an equivalent lock error.

4. Prove that an update remains allowed under CanNotDelete.

```
az tag update `
--resource-id $IdentityId `
--operation Merge `
--tags LockTest=CanUpdate `
-o jsonc
```

Expected result: the update succeeds.

5. Replace the lock with ReadOnly.

```

az lock delete `
  --name "sc500-cannot-delete" `
  --resource-group $RG `
  --resource $IdentityName `
  --resource-type "Microsoft.ManagedIdentity/userAssignedIdentities"

az lock create `
  --name "sc500-read-only" `
  --lock-type ReadOnly `
  --resource-group $RG `
  --resource $IdentityName `
  --resource-type "Microsoft.ManagedIdentity/userAssignedIdentities" `
  --notes "SC-500 Lab 01 read-only test" `
  -o table

```

6. Attempt another tag update.

```

az tag update `
  --resource-id $IdentityId `
  --operation Merge `
  --tags LockTest=ShouldFail `
  -o jsonc

```

Expected result: the write is blocked by the ReadOnly lock.

7. Return to CanNotDelete for the remainder of the lab.

```

az lock delete `
  --name "sc500-read-only" `
  --resource-group $RG `
  --resource $IdentityName `
  --resource-type "Microsoft.ManagedIdentity/userAssignedIdentities"

az lock create `
  --name "sc500-cannot-delete" `
  --lock-type CanNotDelete `
  --resource-group $RG `
  --resource $IdentityName `
  --resource-type "Microsoft.ManagedIdentity/userAssignedIdentities" `
  --notes "SC-500 Lab 01 deletion test" `
  -o table

```

8. Portal verification:

- Open the managed identity.
- Select **Settings > Locks**.
- Confirm the lock type is **Delete**.
- Return to the resource group and select **Locks**.
- Observe that a lock placed directly on the identity is not a resource-group lock.

Checkpoint 4

Expected results:

- Delete fails under CanNotDelete.
- Tag update succeeds under CanNotDelete.
- Tag update fails under ReadOnly.
- The final lock is CanNotDelete.

1.5 Manage built-in Azure role assignments and remediate overprivilege

Azure RBAC authorization is the combination of security principal, role definition, and scope. Narrow scope and minimum permissions are the central design goals.

1.5.1 Prepare a storage data-plane test object

1. Assign the signed-in administrator a temporary data-plane role. Subscription Owner does not automatically grant blob data access.

```
az role assignment create `
  --assignee-object-id $CurrentUserObjectId `
  --assignee-principal-type User `
  --role "Storage Blob Data Contributor" `
  --scope $StorageId `
  -o table
```

2. Wait for RBAC propagation.

```
Start-Sleep -Seconds 60
```

3. Create a private container.

```
az storage container create `
  --account-name $StorageName `
  --name evidence `
  --auth-mode login `
  -o table
```

4. Create and upload an evidence file.

```
"SC-500 Lab 01 governance evidence" |
  Set-Content -Path ".\governance-evidence.txt" -Encoding utf8

az storage blob upload `
  --account-name $StorageName `
  --container-name evidence `
  --name governance-evidence.txt `
  --file ".\governance-evidence.txt" `
  --auth-mode login `
  --overwrite `
  -o table
```

1.5.2 Assign built-in roles to the test user

1. Assign Reader at resource-group scope.

```
az role assignment create `
  --assignee-object-id $TestUserId `
  --assignee-principal-type User `
  --role Reader `
  --scope $RgId `
  -o table
```

2. Assign Storage Blob Data Reader only at the storage-account scope.

```
az role assignment create `
  --assignee-object-id $TestUserId `
  --assignee-principal-type User `
  --role "Storage Blob Data Reader" `
  --scope $StorageId `
  -o table
```

3. List effective direct assignments in the lab scope.

```
az role assignment list `
  --assignee-object-id $TestUserId `
  --scope $RgId `
  --all `
  --include-inherited `
  --query "[].{Role:roleDefinitionName,Scope:scope,PrincipalType:principalType}" `
  -o table
```

4. Open an InPrivate browser session and sign in as the test user.

Validate:

- The test user can open the lab resource group and view resources.
- The test user cannot change a resource tag or storage configuration.
- In the storage account, the test user can browse the evidence container and read the blob.
- The ability to read resource configuration comes from Reader.
- The ability to read blob content comes from Storage Blob Data Reader.

5. Simulate overprivilege by assigning Contributor at resource-group scope.

```
az role assignment create `
--assignee-object-id $TestUserId `
--assignee-principal-type User `
--role Contributor `
--scope $RgId `
-o table
```

6. Re-run the assignment inventory.

```
az role assignment list `
--assignee-object-id $TestUserId `
--scope $RgId `
--all `
--include-inherited `
--query "[].{Role:roleDefinitionName,Scope:scope}" `
-o table
```

The Contributor assignment is overprivileged for a user whose function is governance inspection.

7. Remove Contributor immediately.

```
az role assignment delete `
--assignee-object-id $TestUserId `
--role Contributor `
--scope $RgId
```

8. Confirm remediation.

```
az role assignment list `
--assignee-object-id $TestUserId `
--scope $RgId `
--all `
--include-inherited `
--query "[].{Role:roleDefinitionName,Scope:scope}" `
-o table
```

Expected result: Contributor is absent.

1.5.3 Create and assign an Azure custom role

1. Create a role definition. AssignableScopes controls where the definition can be assigned. It does not itself grant access.

```
$AzureRoleDefinition = @{
    Name = $AzureCustomRoleName
    Id = (New-Guid).Guid
    IsCustom = $true
    Description = "Reads resource inventory, storage configuration, diagnostic settings, and
authorization metadata. No writes and no data actions."
    Actions = @(
        "Microsoft.Resources/subscriptions/resourceGroups/read",
        "Microsoft.Resources/subscriptions/resourceGroups/resources/read",
        "Microsoft.Resources/subscriptions/resources/read",
        "Microsoft.Storage/storageAccounts/read",
        "Microsoft.Authorization/roleAssignments/read",
        "Microsoft.Authorization/roleDefinitions/read",
        "Microsoft.Insights/diagnosticSettings/read"
    )
}
```

```

    )
    NotActions = @()
    DataActions = @()
    NotDataActions = @()
    AssignableScopes = @(
        $SubscriptionScope
    )
}

$AzureRoleDefinition |
    ConvertTo-Json -Depth 10 |
    Set-Content -Path ".\azure-governance-auditor-role.json" -Encoding utf8

```

2. Create the role.

```

az role definition create `
  --role-definition ".\azure-governance-auditor-role.json" `
  -o jsonc

```

3. Assign the custom role at resource-group scope.

```

az role assignment create `
  --assignee-object-id $TestUserId `
  --assignee-principal-type User `
  --role $AzureCustomRoleName `
  --scope $RgId `
  -o table

```

4. Remove the broader built-in Reader assignment.

```

az role assignment delete `
  --assignee-object-id $TestUserId `
  --role Reader `
  --scope $RgId

```

5. Retest as the test user after propagation.

Expected result:

- The user can inspect the resource group and storage-account configuration covered by the custom role.
- The user cannot modify resources.
- The user retains blob-read access because Storage Blob Data Reader remains assigned at the storage-account scope.
- The Azure custom role has no DataActions, so it does not grant access to blob contents.

6. Portal verification:

- Open **Subscription > Access control (IAM) > Roles**.
- Filter **Type = CustomRole**.
- Open the custom role and review **Permissions** and **Assignable scopes**.
- Open the lab resource group > **Access control (IAM) > Role assignments**.
- Filter by the test user.
- Confirm the custom role and the separate Storage Blob Data Reader role.

Checkpoint 5

Record a before-and-after table:

State	Control-plane role	Data-plane role	Result
Initial	Reader at resource group	Storage Blob Data Reader at storage	Read configuration and blob data
Overprivileged	Contributor at resource group	Storage Blob Data Reader at storage	Unnecessary write capability

State	Control-plane role	Data-plane role	Result
Remediated	Custom Governance Auditor at resource group	Storage Blob Data Reader at storage	Narrow inspection plus explicit blob read

1.6 Create and assign a Microsoft Entra custom role

Azure roles manage Azure resources. Microsoft Entra roles manage directory objects. These role systems are separate even though both use the term RBAC.

This exercise creates an app registration, a custom role that can update only basic app-registration properties, and an assignment scoped to that single app registration.

1. Install Microsoft Graph PowerShell if needed.

```
Install-Module Microsoft.Graph.Authentication -Scope CurrentUser -Force
Install-Module Microsoft.Graph.Applications -Scope CurrentUser -Force
Install-Module Microsoft.Graph.Identity.Governance -Scope CurrentUser -Force
Install-Module Microsoft.Graph.Users -Scope CurrentUser -Force
```

2. Connect with the required delegated permissions.

```
Connect-MgGraph -TenantId $TenantId -Scopes `
  "Application.ReadWrite.All", `
  "RoleManagement.ReadWrite.Directory", `
  "User.Read.All"

Get-MgContext | Format-List Account,TenantId,Scopes
```

3. Resolve the test user.

```
$EntraTestUser = Get-MgUser -UserId $TestUserUpn
$EntraTestUser | Select-Object Id,DisplayName,UserPrincipalName
```

4. Create an app registration for scoped delegation.

```
$ScopedApp = New-MgApplication -DisplayName "$Prefix-role-test-app"
$ScopedApp | Select-Object Id,AppId,DisplayName
```

5. Define the custom role.

```
$RolePermissions = @{
    allowedResourceActions = @(
        "microsoft.directory/applications/basic/update"
    )
}

$EntraCustomRole = New-MgRoleManagementDirectoryRoleDefinition `
  -DisplayName $EntraCustomRoleName `
  -Description "Can update basic properties of the scoped app registration. Cannot update credentials." `
  -TemplateId (New-Guid).Guid `
  -RolePermissions $RolePermissions `
  -IsEnabled

$EntraCustomRole | Select-Object Id,DisplayName,Description,IsEnabled
```

6. Assign the role only to the app registration.

```
$AppDirectoryScope = "/$(($ScopedApp.Id))"

$EntraRoleAssignment = New-MgRoleManagementDirectoryRoleAssignment `
  -PrincipalId $EntraTestUser.Id `
  -RoleDefinitionId $EntraCustomRole.Id `
  -DirectoryScopeId $AppDirectoryScope
```

\$EntraRoleAssignment Select-Object Id,PrincipalId,RoleDefinitionId,DirectoryScopeId

7. Portal verification:

- Open the **Microsoft Entra admin center**.
- Browse to **Entra ID > Roles & admins**.
- Search for the custom role.
- Review its single allowed permission.
- Browse to **Entra ID > App registrations > All applications**.
- Open the app named <prefix>-role-test-app.
- Open **Roles and administrators**.
- Open the custom role and confirm that the test user has an assignment at **This resource** scope.

8. Test with the test user in an InPrivate session:

- Open the scoped app registration.
- Change a basic property such as **Home page URL** or **Notes**, if exposed in the current portal experience.
- Attempt to add a client secret or certificate.

Expected results:

- A basic-property update succeeds.
- Credential management is denied because the role lacks `microsoft.directory/applications/credentials/update`.
- The same role does not grant rights over other app registrations.

If Microsoft Entra ID P1/P2 is unavailable, document the licensing blocker and perform the role-design exercise in the portal without selecting **Create**. Do not substitute a broad built-in application administrator role.

Checkpoint 6

Record:

- Custom Microsoft Entra role definition ID.
- App registration object ID.
- Assignment scope.
- Successful basic-property update.
- Denied credential update.

1.7 Evaluate Defender for Cloud security standards, recommendations, and regulatory compliance

Defender for Cloud automatically provides Foundational CSPM capabilities for Azure subscriptions. The Microsoft Cloud Security Benchmark is the default benchmark. Advanced Defender CSPM capabilities are paid.

1. Open **Azure portal > Microsoft Defender for Cloud**.
2. Open **Management > Environment settings**.
3. Select the lab subscription.
4. Open **Defender plans**.
5. Verify:
 - Foundational CSPM is available.
 - Defender CSPM is not being enabled by this lab.
 - No paid workload-protection plan is turned on solely for this exercise.

If the subscription already has paid plans for another purpose, do not change them without authorization.

6. Open **Security policies** for the subscription.
7. Review the **Microsoft Cloud Security Benchmark** assignment.
8. Open the benchmark and inspect:
 - Assigned scope.
 - Included policy initiative.
 - Parameterization.
 - Disabled or exempted controls, if any.
 - Compliance effect of the policies.
9. Open **Defender for Cloud > Regulatory compliance**.
10. Select the Microsoft Cloud Security Benchmark.
11. Inspect at least one control in each available state:
 - Passed.
 - Failed.
 - Not applicable.
 - Shared responsibility or manual, if displayed.
12. For one failed or unhealthy control, record:
 - Control name.
 - Assessment or recommendation.
 - Affected resource.
 - Severity.
 - Remediation guidance.
 - Whether the control is preventive, detective, or both.
13. Open **Defender for Cloud > Recommendations**.
14. Filter to the lab subscription and lab resource group.
15. Open a recommendation affecting the storage account or governance posture. If the resource has not yet been evaluated, record the evaluation delay and use **Policy > Compliance** as the immediate technical evidence.
16. In the recommendation details, inspect:
 - Risk factors and severity.
 - Exposed resources.
 - Remediation steps.
 - Exemptions.
 - Azure Policy relationship.
 - Fix or remediation workflow, if available.
17. Optional cross-portal validation:
 - Open **Microsoft Defender portal**.
 - Browse to **Exposure Management > Recommendations > Cloud**.
 - Compare the consumption-focused view with the Azure portal configuration view.
18. Do not create a paid custom security standard. Instead, identify where the current portal exposes **Add standard** or custom-standard functionality and record the licensing requirement.

Checkpoint 7

Capture:

- MCSB assignment at the subscription.
- One control and its compliance state.

- One Defender for Cloud recommendation and its affected resource.
- Confirmation that no paid Defender plan was enabled by the lab.

1.8 Configure Azure Backup security controls

The lab creates an empty Recovery Services vault. No workload is protected, so there is no backup storage consumption.

1.8.1 Create and harden the Recovery Services vault

1. Create the vault with reversible immutability.

```
az backup vault create `
  --resource-group $RG `
  --name $VaultName `
  --location $Location `
  --immutability-state Unlocked `
  --public-network-access Enable `
  --tags `
    Environment=Lab `
    Workload=SC-500 `
    DataClassification=Internal `
  -o table
```

Unlocked means immutable-vault protection is enabled but can still be reversed. Do not set Locked in a disposable lab.

2. Set locally redundant backup storage before protecting any data.

```
az backup vault update `
  --resource-group $RG `
  --name $VaultName `
  --backup-storage-redundancy LocallyRedundant `
  -o table
```

3. Set soft delete to AlwaysOn with 14-day retention.

```
az backup vault backup-properties set `
  --resource-group $RG `
  --name $VaultName `
  --soft-delete-feature-state AlwaysOn `
  --soft-delete-duration 14 `
  -o jsonc
```

In current Azure Backup behaviour, soft delete may already be enforced or displayed as always-on. Do not attempt to disable it.

4. Inspect the settings.

```
az backup vault show `
  --resource-group $RG `
  --name $VaultName `
  --query "{
    Name:name,
    Location:location,
    Immutability:properties.securitySettings.immutabilitySettings.state,
    PublicNetworkAccess:properties.publicNetworkAccess,
    Redundancy:properties.redundancySettings.standardTierStorageRedundancy
  }" `
  -o jsonc

az backup vault backup-properties show `
  --resource-group $RG `
  --name $VaultName `
  -o jsonc
```

5. Portal verification:

- Open **Recovery Services vaults** > <VaultName>.
- Open **Properties**.
- Inspect **Security Settings**.
- Confirm soft delete is enabled or always-on.
- Confirm **Immutable vault** is enabled but not locked.
- Confirm storage replication is **Locally-redundant**.
- Do not configure a backup item.

1.8.2 Create a Resource Guard and enable multi-user authorization

For a production design, the Resource Guard should be managed by a separate security authority, preferably in another subscription or tenant. This lab keeps it in a separate resource group in the same subscription to avoid extra tenancy requirements. That demonstrates configuration, not true separation of duties.

1. Create the separate Resource Guard resource group.

```
az group create `
--name $GuardRG `
--location $Location `
--tags `
    Environment=Lab `
    Workload=SC-500 `
    SecurityBoundary=ResourceGuard `
    DataClassification=Internal `
-o table
```

2. Ensure the required Azure CLI extension is installed.

```
az extension add --name dataprotection --upgrade
```

3. Create the Resource Guard.

```
az dataprotection resource-guard create `
--resource-group $GuardRG `
--resource-guard-name $ResourceGuardName `
--location $Location `
--tags `
    Environment=Lab `
    Workload=SC-500 `
    SecurityBoundary=ResourceGuard `
-o jsonc
```

4. Capture the Resource Guard ID.

```
$ResourceGuardId = az dataprotection resource-guard show `
--resource-group $GuardRG `
--resource-guard-name $ResourceGuardName `
--query id -o tsv

$ResourceGuardId
```

5. Review protected operations for Recovery Services vaults.

```
az dataprotection resource-guard list-protected-operations `
--resource-group $GuardRG `
--resource-guard-name $ResourceGuardName `
--resource-type "Microsoft.RecoveryServices/vaults" `
-o jsonc
```

6. Associate the Resource Guard with the Recovery Services vault.

```
az backup vault resource-guard-mapping update `
--resource-group $RG `
--name $VaultName `
--resource-guard-id $ResourceGuardId `
-o jsonc
```

7. Verify the mapping.

```
az backup vault resource-guard-mapping show `
--resource-group $RG `
--name $VaultName `
-o jsonc
```

8. Portal verification:

- Open the Recovery Services vault.
- Open **Properties > Multi-User Authorization**.
- Confirm that the vault is protected by the Resource Guard.
- Open the Resource Guard.
- Review **Protected operations**.
- Review **Access control (IAM)**.

9. Document the production separation model:

- Backup administrators manage the vault.
- Security administrators manage the Resource Guard.
- Backup administrators receive only the Resource Guard access needed to request or perform approved critical operations.
- They must not hold broad write permissions over the Resource Guard.
- High-impact actions such as disabling soft delete, disabling MUA, changing protected operations, or deleting protection require authorization through the Resource Guard boundary.

Checkpoint 8

Expected results:

- Vault is empty.
- Soft delete is enabled or always-on.
- Immutability is Unlocked, never Locked.
- Backup storage redundancy is LRS.
- Resource Guard mapping exists.
- No backup policy or protected item exists.

1.9 Final implementation evidence

Before proceeding to theory and assessment, confirm all of the following:

Evidence	Expected state
Bicep validation	Succeeded
Bicep what-if	Only lab-scoped resources and policy objects
Custom Audit policy before fix	Storage account NonCompliant
Custom Audit policy after fix	Storage account Compliant
Modify policy	GovernedBy=SC-500-Lab
Deny policy	Untagged identity blocked
CanNotDelete lock	Delete blocked, update allowed
ReadOnly lock test	Update blocked
Test user overprivilege	Contributor assigned, identified, removed
Azure custom role	Read-only management actions, no DataActions
Entra custom role	Basic app update only, app-scoped
Defender for Cloud	MCSB and at least one recommendation reviewed
Recovery Services vault	Soft delete, reversible immutability, MUA
Paid plan status	No paid Defender plan enabled by this lab

2. Security context

2.1 Governance control stack

The controls in this lab operate at different layers:

1. **Infrastructure as code** defines an intended state in source-controlled, reviewable form.
2. **Azure Policy** evaluates or enforces resource properties across a scope.
3. **Azure RBAC** controls which principals can perform management-plane and data-plane operations.
4. **Microsoft Entra roles** control directory administration rather than Azure resource administration.
5. **Resource locks** add a management-plane protection against update or deletion.
6. **Defender for Cloud** converts configuration and threat intelligence into posture recommendations, standards, and compliance views.
7. **Azure Backup security features** protect recovery capability from administrative error and destructive compromise.

These controls are complementary. None replaces the others.

2.2 Infrastructure as code

Bicep provides a declarative resource model. The template states the desired configuration and Azure Resource Manager determines the deployment sequence. `validate` detects template and authorization problems that can be evaluated before deployment. `what-if` previews additions, changes, and deletions.

The security value is not merely automation. A reviewed template creates:

- Reproducibility.
- Change history.
- Peer-review opportunities.
- Standardized secure defaults.
- A deployment artifact that can be tested before production.
- A basis for policy-as-code and CI/CD gates.

A Bicep template is not automatically secure. An insecure property in a template is simply an insecure configuration deployed consistently. Policy and security review still matter.

2.3 Azure Policy

A policy definition contains a condition and effect. An assignment applies that definition or initiative to a scope.

The lab used three effects:

- **Audit** detected a storage account that allowed shared-key authorization.
- **Deny** rejected creation when a required classification tag was absent.
- **Modify** added or corrected a governance tag.

Modify and DeployIfNotExists policies need a managed identity for remediation because Azure Policy must perform write operations on resources. The identity needs only the role required for the remediation action. Existing resources require an explicit remediation task unless another evaluation and remediation mechanism changes them.

An initiative is a collection of policy definitions organized under a shared objective. Defender for Cloud standards commonly use Azure Policy initiatives underneath their compliance views.

2.4 Azure RBAC and Microsoft Entra RBAC

Azure RBAC protects Azure resources. A role assignment combines:

- Security principal.
- Role definition.
- Scope.

A role definition's `AssignableScopes` determines where the role can be assigned. The actual access granted is controlled by the assignment scope.

Management actions are listed under `Actions`. Data-plane operations are listed under `DataActions`. This explains why a subscription Owner can administer a storage account but still require a blob data role to read blob contents through Microsoft Entra authorization.

Microsoft Entra roles protect directory objects such as users, groups, app registrations, enterprise applications, and consent configuration. An Azure Contributor assignment does not make a principal an Application Administrator, and an Application Administrator assignment does not make a principal an Azure Contributor.

2.5 Resource locks

A lock is an ARM management-plane control:

- **CanNotDelete** blocks deletion but allows authorized modifications.
- **ReadOnly** blocks modifications and deletion.

Locks inherit from parent scopes. A resource-group lock affects child resources. A lock does not replace RBAC because the principal still needs authorization before the lock is evaluated. A sufficiently privileged principal can remove a lock and then perform the blocked action, so lock deletion must be monitored.

Locks also do not protect all data-plane operations. For example, locking a storage account does not make every blob immutable.

2.6 Defender for Cloud

Foundational CSPM provides posture recommendations and the Microsoft Cloud Security Benchmark. Defender CSPM adds paid capabilities such as advanced governance, attack-path analysis, cloud security explorer, and other enriched posture functions.

Security standards organize controls. Recommendations identify resource-level corrective actions. The regulatory compliance experience aggregates assessments against assigned standards. The Azure portal is the configuration plane for standards, while the Microsoft Defender portal provides a unified consumption and tracking experience.

A compliance dashboard is not an attestation that the organization meets every legal or regulatory obligation. It is technical evidence mapped to controls, with shared-responsibility and manual requirements still requiring organizational validation.

2.7 Azure Backup security

Backup security addresses a different failure mode from resource prevention. An attacker may obtain valid administrative access and then attempt to erase recovery points before encrypting or deleting production resources.

The key controls are:

- **Soft delete** retains deleted backup data for a recovery window.
- **Immutable vault** prevents operations that would remove recovery points before retention expires.
- **Resource Guard and MUA** require authorization through a separate control boundary for critical operations.
- **Separate administration** prevents one compromised administrative identity from controlling both production and the recovery safeguard.

For production, use a Resource Guard controlled by a separate team, subscription, or tenant. The same-subscription lab topology proves the feature mechanics but does not provide strong separation of duties.

3. Threat perspective

3.1 Threat model

A realistic cloud governance attack can proceed as follows:

1. The adversary obtains a valid cloud account.
2. The adversary enumerates role assignments and finds excessive privilege.
3. The adversary adds or activates another privileged role to persist access.
4. The adversary weakens policy, removes locks, or deploys resources outside approved controls.
5. The adversary disables backup protections or deletes recovery assets.
6. The adversary deletes or encrypts production data.

3.2 MITRE ATT&CK mapping

ATT&CK technique	Adversary behaviour	Relevant controls from this lab	Limitation
T1078.004 - Valid Accounts: Cloud Accounts	Uses compromised cloud credentials for legitimate-looking administrative operations	Least privilege, narrow scopes, role review, Defender recommendations, Activity Log monitoring	These controls do not prevent use of a valid account that still has authorized access
T1098.003 - Account Manipulation: Additional Cloud Roles	Adds privileged cloud roles to increase access or persistence	RBAC inventory, removal of Contributor, custom roles, separate Entra and Azure role review	A highly privileged attacker may still create assignments unless protected by process, PIM, alerting, and separation
T1098.001 - Account Manipulation: Additional Cloud Credentials	Adds app secrets, certificates, or credentials	App-scoped Entra custom role excludes credential-update permissions	Another role with credential permissions can still add credentials
T1490 - Inhibit System Recovery	Disables or deletes recovery mechanisms before impact	Soft delete, reversible immutability for the lab, production locked immutability, Resource Guard, MUA, separate backup administration	Lab immutability remains reversible for cleanup and is not production strength
T1578.003 - Modify Cloud Compute Infrastructure: Delete Cloud Instance	Deletes cloud resources to cause impact or remove evidence	Deny policies, CanNotDelete locks, least privilege, monitored role and lock changes	Locks can be removed by an authorized principal; deletion monitoring remains necessary

3.3 How the controls interrupt the attack

- The custom role reduces the number of write operations available to the test principal.
- Separate data-plane roles prevent a management-plane role from automatically exposing stored data.
- Deny policy blocks known-bad deployment state before resource creation.
- Modify policy repairs supported configuration drift and tags existing resources through remediation.
- Audit policy provides evidence when a property does not meet the standard.
- Resource locks force an attacker to perform an additional, visible control-plane action before deletion.
- Defender for Cloud consolidates posture findings and maps them to standards.

- Resource Guard adds a second authorization boundary around critical backup operations.
- Soft delete and immutability preserve recovery options after deletion attempts.

3.4 High-value telemetry

Monitor these operation families in Azure Activity Log and Microsoft Entra audit logs:

```
Microsoft.Authorization/roleAssignments/write
Microsoft.Authorization/roleAssignments/delete
Microsoft.Authorization/policyAssignments/write
Microsoft.Authorization/policyAssignments/delete
Microsoft.Authorization/locks/write
Microsoft.Authorization/locks/delete
Microsoft.RecoveryServices/vaults/write
Microsoft.RecoveryServices/vaults/delete
Microsoft.DataProtection/resourceGuards/write
Microsoft.DataProtection/resourceGuards/delete
Microsoft.Storage/storageAccounts/write
```

Also monitor:

- Creation and deletion of custom Azure role definitions.
- Creation and deletion of Microsoft Entra custom roles.
- App registration credential changes.
- Defender for Cloud plan changes.
- Policy exemptions.
- Backup protection stops and delete-data operations.
- Resource Guard mapping changes.

4. Self-assessment

Do not use the answer section until all questions are complete.

Question 1 - Single answer

A management group contains 40 subscriptions. Existing resources lack the tag `CostCenter=S0C`. New and updated resources must receive the tag automatically, and existing resources must be corrected without recreation.

Which solution should you implement?

- A. Assign an Audit policy and trigger a compliance scan.
- B. Assign a Deny policy and create an exemption for existing resources.
- C. Assign a Modify policy with a managed identity, grant the required tag role, and create a remediation task.
- D. Apply a CanNotDelete lock at the management-group scope.

Question 2 - Select all that apply

You create an Azure custom role with:

```
Actions:
  Microsoft.Storage/storageAccounts/read

DataActions:
  None

AssignableScopes:
  /subscriptions/Sub1
```

You assign the role to User1 at resource group RG1. A storage account in RG1 contains private blobs.

Which statements are true? Select all that apply.

- A. User1 can read storage-account configuration in RG1.
- B. User1 can read private blob contents because the role is assignable at subscription scope.
- C. The role can be assigned at RG1 because RG1 is beneath an AssignableScope.
- D. User1 can modify the storage-account firewall.
- E. User1 needs a storage data-plane role to read blob contents.

Question 3 - Ordering

Arrange the steps for a controlled Bicep deployment in the correct order.

1. Run the deployment.
2. Run `what-if`.
3. Author or modify the Bicep files.
4. Validate the deployment.
5. Review deployed outputs and policy compliance.

Write the order as a sequence of numbers.

Question 4 - Yes or No

For each statement, answer Yes or No.

1. A CanNotDelete lock prevents an authorized user from changing resource tags.
2. A ReadOnly lock prevents an authorized user from deleting the locked resource.
3. A user with sufficient lock-management permissions can remove a lock and then delete the resource.
4. A storage-account resource lock makes existing blobs immutable.

Question 5 - Single answer

A developer must update the display name and home-page URL of one app registration. The developer must not create secrets or modify any other app registration.

What should you configure?

- A. Azure Contributor at the app registration's resource-group scope.
- B. Microsoft Entra Application Administrator at tenant scope.
- C. A Microsoft Entra custom role containing `microsoft.directory/applications/basic/update`, assigned at the specific app-registration scope.
- D. An Azure custom role with `Microsoft.Authorization/*/read`.

Question 6 - Select all that apply

A security engineer is reviewing Microsoft Defender for Cloud without enabling a paid plan.

Which capabilities are expected from Foundational CSPM? Select all that apply.

- A. Microsoft Cloud Security Benchmark recommendations.
- B. Secure score based on applicable recommendations.
- C. Every advanced attack path and cloud security explorer feature.
- D. Resource-level posture recommendations.
- E. Automatic legal certification that the company complies with every mapped regulation.

Question 7 - Single answer

An organization wants to prevent a compromised backup administrator from disabling soft delete and deleting backup data without a second security authority.

Which control most directly implements the second authorization boundary?

- A. Locally redundant vault storage.
- B. Resource Guard with multi-user authorization, managed by a separate security authority.
- C. A Reader role on the Recovery Services vault.
- D. A CanNotDelete lock on the protected virtual machine only.

Question 8 - Case study

You manage subscription Sub1. A test user needs to:

- Inspect resource inventory in RG1.
- Read one blob container in Storage1.
- Never modify Azure resources.
- Never read blobs in other storage accounts.

The user currently has Contributor at RG1.

Which sequence best meets the requirement?

- A. Keep Contributor and add Reader at Storage1.
- B. Remove Contributor, assign Reader or a constrained custom read role at RG1, and assign Storage Blob Data Reader at Storage1.
- C. Remove Contributor and assign Storage Blob Data Contributor at the subscription.
- D. Assign Global Reader in Microsoft Entra ID and remove all Azure roles.

Question 9 - Ordering

Arrange the policy lifecycle in the correct order for an existing non-compliant resource.

1. Create a remediation task.
2. Assign the policy at the intended scope.
3. Author or select the policy definition.
4. Grant the policy assignment identity the minimum remediation role.
5. Verify the resource is compliant.

Write the order as a sequence of numbers.

Question 10 - Multi-step scenario

A storage account allows shared-key authorization. A custom Audit policy identifies the issue. The account is protected by a CanNotDelete lock.

Which action makes the resource compliant without removing the lock?

- A. Delete and recreate the storage account with shared-key access disabled.
- B. Change `allowSharedKeyAccess` to false.
- C. Replace the lock with ReadOnly and run a policy scan.
- D. Assign Storage Blob Data Owner to the policy identity.

Answers and explanations

Answer 1

C

Modify can add or replace supported tags during resource operations. Existing resources are corrected through a remediation task. The assignment's managed identity requires the minimum role needed to write tags.

Audit only reports. Deny does not repair existing resources. A resource lock does not supply or enforce tag values.

Answer 2

A, C, E

- A is correct because the role grants the storage-account read management action.
- C is correct because RG1 is a child scope of Sub1.
- E is correct because blob content is a data-plane operation and the custom role has no DataActions.
- B is incorrect because AssignableScopes does not grant access and no blob DataAction exists.
- D is incorrect because the role contains no write action.

Answer 3

3, 4, 2, 1, 5

Author the template, validate it, preview the change set, deploy it, then inspect outputs and compliance.

Answer 4

1. **No** - CanNotDelete allows authorized updates.
2. **Yes** - ReadOnly blocks updates and deletion.
3. **Yes** - A sufficiently privileged principal can remove the lock first.
4. **No** - an ARM lock is not blob immutability.

Answer 5

C

The requirement concerns a Microsoft Entra app-registration object, not an Azure resource. A custom Entra role with only the basic-update permission and an app-specific scope meets least privilege. Application Administrator is too broad and can manage credentials.

Answer 6

A, B, D

Foundational CSPM supplies baseline posture management, including MCSB recommendations and secure-score inputs. Advanced attack-path and explorer capabilities are associated with Defender CSPM. A technical dashboard does not constitute automatic legal certification.

Answer 7

B

Resource Guard and MUA require applicable authorization through a separate resource boundary for protected backup operations. Strong production design places control of the Resource Guard with a different security authority.

Answer 8

B

The management-plane requirement is read-only at RG1. The data-plane requirement is read-only at one storage account. Separate assignments at the narrow scopes satisfy both. Contributor and subscription-level data roles are overprivileged. Microsoft Entra Global Reader does not grant Azure resource access.

Answer 9

3, 2, 4, 1, 5

Choose or author the definition, assign it, authorize the assignment identity, remediate the existing resource, then verify compliance.

Answer 10

B

CanNotDelete permits authorized updates. Disabling shared-key authorization changes the non-compliant property without deleting the account. ReadOnly would block the update. Blob data ownership is unrelated to the storage-account management property.

5. Cleanup and cost sanity check

Cleanup order matters. Remove locks and protected mappings before deleting resources.

5.1 Return to the administrator context

If you switched to the test user, sign out and authenticate as the lab administrator.

```
az logout
az login --use-device-code
az account set --subscription $SubscriptionId
```

Reconnect to Microsoft Graph as the privileged administrator if the Graph session expired.

```
Connect-MgGraph -TenantId $TenantId -Scopes `
  "Application.ReadWrite.All", `
  "RoleManagement.ReadWrite.Directory", `
  "User.Read.All"
```

5.2 Remove the managed identity lock

```
az lock delete `
  --name "sc500-cannot-delete" `
  --resource-group $RG `
  --resource $IdentityName `
  --resource-type "Microsoft.ManagedIdentity/userAssignedIdentities"
```

Verify no direct lock remains.

```
az lock list `
  --resource-group $RG `
  --resource $IdentityName `
  --resource-type "Microsoft.ManagedIdentity/userAssignedIdentities" `
  -o table
```

5.3 Remove Microsoft Entra custom-role objects

Use the variables retained from Section 1.6.

```
if ($EntraRoleAssignment.Id) {
    Remove-MgRoleManagementDirectoryRoleAssignment `
        -UnifiedRoleAssignmentId $EntraRoleAssignment.Id
}

if ($EntraCustomRole.Id) {
    Remove-MgRoleManagementDirectoryRoleDefinition `
        -UnifiedRoleDefinitionId $EntraCustomRole.Id
}

if ($ScopedApp.Id) {
    Remove-MgApplication -ApplicationId $ScopedApp.Id
}

Disconnect-MgGraph
```

If the shell was closed, resolve objects by display name before removal:

```
Connect-MgGraph -TenantId $TenantId -Scopes `
    "Application.ReadWrite.All", `
    "RoleManagement.ReadWrite.Directory"

$Role = Get-MgRoleManagementDirectoryRoleDefinition `
    -Filter "displayName eq '$EntraCustomRoleName'"

$Assignments = Get-MgRoleManagementDirectoryRoleAssignment `
    -Filter "roleDefinitionId eq '$($Role.Id)'"

foreach ($Assignment in $Assignments) {
    Remove-MgRoleManagementDirectoryRoleAssignment `
        -UnifiedRoleAssignmentId $Assignment.Id
}

if ($Role) {
    Remove-MgRoleManagementDirectoryRoleDefinition `
        -UnifiedRoleDefinitionId $Role.Id
}

$App = Get-MgApplication -Filter "displayName eq '$Prefix-role-test-app'"
if ($App) {
    Remove-MgApplication -ApplicationId $App.Id
}

Disconnect-MgGraph
```

5.4 Remove Azure role assignments and the Azure custom role

Remove only the assignments created by this lab.

```
az role assignment delete `
    --assignee-object-id $TestUserId `
    --role $AzureCustomRoleName `
    --scope $RgId

az role assignment delete `
    --assignee-object-id $TestUserId `
    --role "Storage Blob Data Reader" `
    --scope $StorageId

az role assignment delete `
    --assignee-object-id $TestUserId `
    --role Reader `
    --scope $RgId 2>$null

az role assignment delete `
    --assignee-object-id $TestUserId `
```

```
--role Contributor `
--scope $RgId 2>$null

az role assignment delete `
--assignee-object-id $CurrentUserObjectId `
--role "Storage Blob Data Contributor" `
--scope $StorageId
```

Delete the custom role definition after its assignments are removed.

```
az role definition delete --name $AzureCustomRoleName
```

Verify:

```
az role definition list `
--custom-role-only true `
--query "[?roleName=='$AzureCustomRoleName'].roleName" `
-o tsv
```

Expected result: no output.

5.5 Remove multi-user authorization and backup resources

1. Remove the Resource Guard mapping.

```
az backup vault resource-guard-mapping delete `
--resource-group $RG `
--name $VaultName `
--yes
```

If the command requests authorization against the Resource Guard, complete the sign-in or approval flow. If it fails, use:

- **Recovery Services vault > Properties > Multi-User Authorization > Update**
- Select the Resource Guard directory.
- Authenticate with an identity authorized on the Resource Guard.
- Disable the mapping.

2. Disable reversible immutability. This succeeds only because the vault was left UnLocked.

```
az backup vault update `
--resource-group $RG `
--name $VaultName `
--immutability-state Disabled `
-o table
```

3. Confirm no protected items or backup containers exist.

```
az backup item list `
--resource-group $RG `
--vault-name $VaultName `
--backup-management-type AzureIaaSVM `
-o table
```

Expected result: no items.

4. Delete the empty vault.

```
az backup vault delete `
--resource-group $RG `
--name $VaultName `
--yes
```

5. Delete the Resource Guard.

```
az dataprotection resource-guard delete `
--resource-group $GuardRG `
```



```
--resource-guard-name $ResourceGuardName `
--yes
```

6. Delete the Resource Guard resource group.

```
az group delete --name $GuardRG --yes --no-wait
```

5.6 Remove Azure Policy objects

1. Delete the built-in policy assignments.

```
az policy assignment delete `
--name $TagAssignmentName `
--scope $RgId

az policy assignment delete `
--name $DenyAssignmentName `
--scope $RgId
```

Deleting the Modify assignment also deletes its system-assigned identity. The role assignment created for that identity can take a short time to disappear.

2. Delete the custom policy assignment.

```
az policy assignment delete `
--name $CustomPolicyAssignmentName `
--scope $RgId
```

3. Delete the custom policy definition.

```
az policy definition delete `
--name $CustomPolicyDefinitionName
```

4. Verify no lab policy object remains.

```
az policy assignment list `
--scope $RgId `
--disable-scope-strict-match true `
--query "[?contains(name, '$Prefix')].{Name:name,Scope:scope}" `
-o table

az policy definition list `
--query "[?contains(name, '$Prefix')].{Name:name,DisplayName:displayName}" `
-o table
```

Expected result: no lab-created object.

5.7 Delete the resource group and deployment record

1. Delete the main lab resource group.

```
az group delete --name $RG --yes --no-wait
```

2. Wait until both groups are absent.

```
do {
  Start-Sleep -Seconds 20
  $RemainingGroups = az group exists --name $RG
  Write-Host "$RG exists: $RemainingGroups"
} while ($RemainingGroups -eq "true")
```

3. Delete the subscription deployment record.

```
az deployment sub delete --name $DeploymentName
```

4. Delete local files.

```
Set-Location $HOME
Remove-Item "$HOME\sc500-lab01" -Recurse -Force
```

5.8 Final verification

Run:

```
az resource list `
  --tag Workload=SC-500 `
  --query "[?contains(name, '$Prefix')].{Name:name,Type:type,ResourceGroup:resourceGroup}" `
  -o table

az group list `
  --query "[?contains(name, '$Prefix')].{Name:name,State:properties.provisioningState}" `
  -o table

az backup vault list `
  --query "[?contains(name, '$Prefix')].{Name:name,ResourceGroup:resourceGroup}" `
  -o table

az dataprotection resource-guard list `
  --query "[?contains(name, '$Prefix')].{Name:name,ResourceGroup:resourceGroup}" `
  -o table
```

Expected result: no lab resource.

Portal checks:

1. Open **Cost Management + Billing > Cost Management > Cost analysis**.
2. Set the scope to the lab subscription.
3. Filter by the two lab resource groups.
4. Confirm no running compute, gateway, database, container, or paid Defender plan exists.
5. Open **Microsoft Defender for Cloud > Environment settings > Defender plans**.
6. Confirm that the lab did not leave a paid plan enabled.
7. Open **Policy > Assignments** and search the prefix.
8. Open **Subscription > Access control (IAM) > Role assignments** and search the test user and prefix.

Cost sanity result

A correctly cleaned lab should leave:

- No Azure resource.
- No Recovery Services vault.
- No Resource Guard.
- No custom policy definition or assignment.
- No custom Azure role.
- No Microsoft Entra custom role or test app.
- No lab role assignment.
- No paid Defender plan enabled by the lab.

Charges, if any, should be limited to negligible storage transaction and metadata usage during the brief lab window.

References

- Microsoft Learn, Study guide for Exam SC-500: Implementing End-to-End Security Controls for Cloud and AI Workloads.
- Microsoft Learn, Overview of Azure Policy.
- Microsoft Learn, Remediate non-compliant resources with Azure Policy.

- Microsoft Learn, Azure Policy Modify effect.
- Microsoft Learn, Azure RBAC custom roles and role assignments.
- Microsoft Learn, Create a custom role in Microsoft Entra ID.
- Microsoft Learn, Regulatory compliance and security policies in Defender for Cloud.
- Microsoft Learn, Cloud Security Posture Management in Defender for Cloud.
- Microsoft Learn, Azure Backup security overview.
- Microsoft Learn, Multi-user authorization using Resource Guard.
- MITRE ATT&CK, T1078.004, T1098.001, T1098.003, T1490, and T1578.003.